Pizza 42 + Auth0

Customer Identity, done right.

Amar Nagra · CIAM Specialist Tech Challenge

Agenda

45 minutes total

- **Business problem** — what Pizza 42 asked for, and why
- **Solution architecture** — components, grants, token flow
- **Live demo** — sign up → verify → order → history
- **Implementation walkthrough** — decisions, trade-offs, what I'd do differently
- **Q&A**

PART 1

The business problem

What Pizza 42 asked for

Three stakeholders, one identity platform

Team	Ask	Auth0 answer
Security	Offload credential management — storing passwords is a liability.	Universal Login. SOC 2 / ISO 27001. Pizza 42 never sees a password.
Product	Frictionless & customizable login. Password reset. Social. Passwordless.	Branded Universal Login, social connections (Google), Passkeys (WebAuthn).
Marketing	Enrich customer profiles to drive campaigns.	Social IdP claims + progressive profiling via Actions + `app_metadata` .

> Every Pizza 42 ask maps to a turn-key Auth0 capability. No identity code we have to maintain.

PART 2

Architecture

Auth0 components

Six logical pieces, each with one job

Component	Type	Job
Pizza 42	SPA application	React frontend; triggers Universal Login.
Pizza 42 API	Custom API (resource server)	Defines audience + scope `create:orders` .
Pizza 42 Backend	Machine-to-Machine app	Server-only creds for the Auth0 Management API.
Google connection	Social IdP	One-click signup; pre-verified email.
Database connection	IdP	Email/password + Passkeys.
Post-Login Action	Auth0 Action	Injects order summary into ID token.

System diagram

React SPA
@auth0/auth0-react
Vercel-hosted

Auth0 Tenant
apnagra.us.auth0.com
Universal Login · Actions · Mgmt API

Pizza 42 API
Vercel serverless
`POST /api/orders`

Data flow

1. SPA → Auth0: Authorization Code + PKCE → ID + access tokens (front channel)
2. SPA → API: `POST /api/orders` with `Bearer <access_token>`
3. API → Auth0 JWKS: verify signature, issuer, audience, scope (back channel)
4. API → Mgmt API: Client Credentials → read user → PATCH `app_metadata.orders` (back channel)
5. Next login → Post-Login Action injects `https://pizza42.com/orders` claim into the ID token

OAuth 2.0 grants — chosen and rejected

Grant	Verdict	Why
Authorization Code + PKCE	✓ SPA uses this	Only safe grant for browser apps. PKCE replaces the client secret with a one-time challenge.
Client Credentials	✓ Backend uses this	No user involved. Server can safely hold the M2M secret.
Implicit	✗ Rejected	Deprecated. Tokens in URL fragment → browser history / referer leakage.
ROPC (password)	✗ Rejected	App sees the password — defeats the security team's #1 ask.
Auth Code without PKCE	✗ Rejected	SPA can't safely hold a client secret.
Device Code	✗ N/A	For input-constrained devices (TVs, CLIs). We have a browser.
Refresh Token	⚠ Disabled in PoC	Using silent iframe renewal instead. Would re-enable with rotation in production.

Front channel vs. back channel

Front channel

Browser-visible redirects. Used for **user interaction**.

- SPA → `/authorize` redirect
- Universal Login UI
- Auth code returned in URL

Anything in the URL is observable — that's why only the short-lived auth code travels here, never the tokens.

Back channel

Server-to-server. Used for **secrets and tokens**.

- Token exchange (`/oauth/token`)
- JWKS fetch for verification
- Mgmt API calls from backend
- M2M Client Credentials grant

Tokens are issued and exchanged here so they never appear in the URL.

> Anti-pattern alert: the deprecated *Implicit* grant returned tokens via the front channel. That's why we don't use it.

The three tokens

Token	Format	Consumer	What it's used for
ID token	JWT (OIDC)	The SPA	"Who is the user?" name, email, picture, `email_verified`, custom orders claim
Access token	JWT, RS256	The Pizza 42 API	"What can the user do?" — bearer credential the backend verifies and enforces `create:orders`
Refresh token	Opaque	Auth0 token endpoint	Disabled in PoC. Would let SPA silently obtain new access tokens with rotation.

Why each is a separate token

- **Different audiences.** Mixing identity claims with API authorization in one token breaks the principle that tokens are for one resource.
- **Different lifetimes.** Access tokens are short (~1h). ID tokens persist for a session. Refresh tokens are longer-lived & rotatable.
- **Different sensitivity.** Refresh tokens never leave server-side storage in serious apps.

The two code surfaces — Action + API

A. Action shapes the token. B. API enforces three gates on every request.

A. Post-Login Action — Inject claims into ID Token

Runs on every login & refresh-token exchange.

```
exports.onExecutePostLogin = async (event, api) => {
  const orders = event.user.app_metadata
    ?.orders ?? [];
  const marketing = event.user.user_metadata
    ?.marketingProfile ?? null;

  api.idToken.setCustomClaim(
    "https://pizza42.com/orders", orders);
  api.idToken.setCustomClaim(
    "https://pizza42.com/marketing_profile",
    marketing);
};
```

B. API — three gates, all server-side

Vercel fn + `jose`. Email check is live, not a token claim.

```
// Gate 1: validate JWT (RS256 / JWKS)
const { payload } = await jwtVerify(token, JWKS, {
  issuer: `https://${AUTH0_DOMAIN}/`,
  audience: AUTH0_AUDIENCE,
});

// Gate 2: require create:orders scope
if (!payload.scope.split(" ").includes("create:orders"))
  return res.status(401).send("Missing scope");

// Gate 3: email_verified — LIVE Mgmt API lookup
const user = await getUser(payload.sub, mgmtToken);
if (!user.email_verified)
  return res.status(403).send("Email not verified");

// Save via M2M Mgmt API → app_metadata.orders
await patchUserAppMetadata(payload.sub, {...}, mgmtToken);
```

> Live Mgmt API lookup for `email_verified` means a user who verifies mid-session can order on their next click — no logout required.

PART 3

Live demo

Demo flow

What you'll see in the next ~20 minutes

1. Landing page (logged out) — branded experience, Universal Login entry points
2. Sign up with email/password — show `email_verified: false` in ID token, order button disabled
3. Verify email via link — banner clears, order button enabled
4. Place an order — show backend logs (scope verified, Mgmt API patch)
5. Log out → log back in — order history populated from custom claim in ID token
6. Log in with Google — one-click, pre-verified email, no verification gate
7. Auth0 dashboard tour — API + scope, Post-Login Action, user_metadata

PART 4

Implementation walkthrough

Tech stack & decisions

Frontend

- React + Vite
- `@auth0/auth0-react`
- Tailwind
- Vercel hosted

Backend

- Vercel serverless fn
- `jose` for JWT/JWKS
- Fetch-based Mgmt API client
- In-memory M2M token cache

Auth0

- Universal Login
- Custom API + scope
- M2M app
- Post-Login Action
- Google + Database connections

Key choices

- **Universal Login** over embedded — Pizza 42 never touches passwords.
- **Plain scopes** over RBAC — fewer moving parts; one scope, one route. Easy to upgrade.
- **M2M for Mgmt API** — credential isolation, least privilege, audit clarity.
- **Server-side email check** — client check is UX, API is authority.

Where the PoC departs from production best practice

Honest critique of the spec — what I'd change for a real Pizza 42

PDF says	Why it's not ideal	Better approach
Put **order history** in the ID token	Tokens bloat over time; potentially sensitive data in long-lived signed token; re-fetched on every silent renew	Put only a *summary* in the token (`orderCount` , `lastOrderAt`). Fetch full list from `/api/orders` .
Save orders to user's Auth0 profile	16 KB cap; not queryable; vendor lock-in; Mgmt API rate limits	Own DB as source of truth. Mirror thin summary into `app_metadata` (system-owned, tamper-proof) only if needed in tokens.
(Implicitly) email check on the client	Hostile clients can call the API directly with a valid unverified token	Enforce on the server (we did) — client check is just UX.
Mgmt API on every order	Hot-path latency; rate-limit exposure	Use Actions / webhooks to keep your own DB derived state in sync.

Challenges & learnings

- **Rules are dead — used Actions.** Half the public reference repos still show the deprecated `context.idToken[...] = ...` Rules pattern. Rules EOL Nov 18, 2026, unavailable to new tenants since Oct 2023. I built on `api.idToken.setCustomClaim()` in a Post-Login Action.
- **Orders in `app_metadata`, not `user_metadata`.** Orders are system-owned, not user-editable. `user_metadata` can be exposed to direct SPA editing via `update:current_user_metadata`; `app_metadata` has no equivalent — tamper-proof by design.
- **`email_verified` via Mgmt API, not access token.** The claim isn't in the access token by default. Server-side Mgmt API lookup gives me always-current state — a user who verifies mid-session can order without re-login.
- **Per-app authorization.** SPA needed explicit grant of `create:orders` in API Access tab — otherwise the scope is silently stripped from tokens.
- **Localhost consent prompt.** Auth0 forces user consent on `localhost` as an anti-phishing safeguard, regardless of "skip consent" setting.
- **Custom claim namespacing.** Auth0 strips non-OIDC claims unless namespaced by URL (`https://pizza42.com/orders`).

Production roadmap

Security hardening

- Enable MFA + breached password detection
- Refresh token rotation + reuse detection
- Step-up auth on payment / high-value orders
- Private-key JWT for M2M (replace shared secret)
- Auth0 log streams → SIEM

UX & growth

- Custom domain (`auth.pizza42.com`)
- Account linking (email + Google → one identity)
- Progressive profiling Action — capture marketing data gradually
- Passkeys on by default
- Localized Universal Login per region

Adjacent opportunity: GenAI

- **Reorder bot** grounded on `user_metadata` order history
- **Auth for GenAI** — delegated tool use, agent identity, scoped consent

PART 5

Questions?

Common questions — quick reference

Appendix

Question	Short answer
Why PKCE?	SPA can't safely hold a secret. PKCE protects against auth-code interception.
Why a separate M2M app?	Credential isolation, different grant, least privilege, audit clarity.
Refresh tokens?	Disabled in PoC; silent iframe renewal instead. Would re-enable with rotation in production.
Front vs back channel?	Front = browser-visible redirects. Back = server-to-server. Tokens issued via back channel.
10M users — what changes?	Own DB as source of truth; cache; custom domain; Auth0 log streams → SIEM.
How rotate the M2M secret?	Multi-credential support — generate new, deploy, rotate traffic, revoke old. Or switch to private-key JWT.
Auth0 down?	Cached JWKS keeps JWT validation working. New logins blocked. Multi-IdP fallback is a bigger arch change.
Progressive profiling?	Post-Login Action prompts for marketing data over multiple visits; stored in `user_metadata`.